

Artificial Intelligence in Games

Project - NEAT Application to the Racing Cars Game

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

Group 05

Bruno Faustino fc59784
Rodrigo Neto fc59850

2025/2026

1 Introduction

This project applies **NeuroEvolution of Augmenting Topologies** (NEAT) to the Racing Cars game built with `pygame`. The goal is to evolve neural network controllers that drive an autonomous car around a closed circuit as fast as possible, staying within the track boundaries throughout. NEAT, introduced by Stanley and Miikkulainen (2002), simultaneously evolves the weights *and* the topology of a neural network, starting from a minimal direct connection between inputs and outputs. Structural complexity is added incrementally through mutations that insert new nodes or connections, and speciation protects structural innovations from being eliminated before they have time to optimise. This makes NEAT well suited to continuous control problems such as racing, where the optimal network size is unknown in advance.

Two complementary control strategies are investigated:

- **Waypoints car** — the network receives geometric information about the next waypoint on a pre-defined route and outputs continuous steering and throttle commands.
- **Radar car** — the network receives distances measured by a set of forward-facing radar beams and outputs the same two commands, with no prior knowledge of the track layout.

2 Controller Inputs

2.1 Waypoints Car Inputs

The waypoints car receives a vector of five normalised real-valued inputs at every simulation frame (60 Hz):

Index	Name	Formula / source	Range
0	Distance to waypoint	$\min(d/600, 1)$	$[0, 1]$
1	Signed angle to waypoint	$\alpha/180$	$[-1, 1]$
2	Current curve sharpness	$\kappa/180$	$[0, 1]$
3	Next curve sharpness	$\kappa_{\text{next}}/180$	$[0, 1]$
4	Normalised speed	v/v_{max}	$[-0.5, 1]$

Distance to waypoint. The Euclidean distance from the car’s centre to the centre of the current waypoint, clipped and scaled so that distances beyond 600 px map to 1.0.

Signed angle to waypoint. The signed angular deviation between the car’s heading vector and the direction to the current waypoint, expressed in units of 180° . A positive value indicates the waypoint lies to the left; negative to the right. This is computed using the cross product of the heading unit vector and the waypoint direction vector. Because pygame uses a y-down coordinate system, the raw cross-product sign is flipped relative to screen geometry; the implementation corrects for this by inverting the sign test so that the returned angle matches the visual left/right perception.

Curve sharpness. The exterior angle (in degrees) formed by the path segments at the current and next waypoints, respectively. A value of 0° corresponds to a straight segment; 180° to a U-turn. By providing both the current and next sharpness values the network can anticipate upcoming turns and brake proactively.

Normalised speed. The car’s current velocity divided by its maximum velocity (4 px/frame), giving a value in $[-0.5, 1]$ since the car can reverse at up to half speed.

Sensor and actuator noise. Two utility functions in `noise_utils.py` add Gaussian perturbations to simulate real-world uncertainty:

- **Sensor noise:** $\tilde{s} = s + \mathcal{N}(0, 0.03)$, applied to every input before it reaches the network. Values are *not* clipped, allowing the network to receive out-of-range readings just as a real sensor might drift.
- **Actuator noise:** $\tilde{a} = \text{clip}(a + \mathcal{N}(0, 0.05), -1, 1)$, applied to the steering and throttle outputs before they update the car’s state.

name	angle (degrees)
left	-150
left_middle	-120
middle	90
right_middle	-60
right	-30

An early bug in `add_sensor_noise` clipped all sensor values to $[0, 1]$, which destroyed the signed angle input (range $[-1, 1]$): any left-of-waypoint reading was clamped to 0.0, making the network blind to left turns. This caused the population to collapse to “go straight” behaviour from generation 5 onward. Removing the clip and correcting the cross-product sign in `_signed_angle_to_target` (see above) restored stable learning.

2.2 Radars Car Inputs

The radar car neural network has 5 input nodes, where each node corresponds to one of the radar distances from the car center.

3 Radars

The radar car has 5 radars.

Let’s remember that in `pygame` the angle 0 is at the right and the angles grow clockwise.

The 5 radars are known by their names, angles and distances from the car center: information stored in 2 dictionaries in the `RadarCar` class.

The radars make the shape of an arc in front of the car:

The distance of each radar is set to a maximum amount in the app config file by default. If the radar comes across an obstacle (in this case, the track border), it will be cut off at the obstacle, which will decrease the distance.

Additionally, each radar includes random noise that can change its direction and distance, which is adjustable in the app config file.

4 Analysis

4.1 Waypoints Car Analysis

4.1.1 Experiments

The waypoints controller was developed through several iterations of parameter tuning, documented in the code comments of `neat_car_base.py`. The final values reflect lessons learned from early failures.

Early generation extinction. Initial kill conditions were far more aggressive: 30 consecutive off-track frames, a 10-frame grace period at spawn, and a waypoint-stuck timer of 600 frames. Together with a circular-motion detector that killed any car whose position stayed within 25 px of a previously visited

position for 90 frames, most generation-0 cars died within a second. Because all cars scored near-zero fitness, there was no usable selection gradient and the population stagnated.

Relaxed kill conditions and shortcut exploit. Kill thresholds were progressively relaxed to the current values (90 consecutive off-track frames, 30-frame grace period, 900-frame stuck timer, tighter circular detector at 20 px radius applied over 120 frames). This allowed generation-0 cars to survive long enough to produce a fitness signal.

However, an exploit emerged: the start position (180, 200) and finish line (130, 250) are only ≈ 64 px apart, so a car could drive a tight circle from spawn directly over the finish line and collect the full finish bonus without visiting any waypoints. The root cause was that `_check_finish_line()` originally required no waypoint progress.

Finish-line gate (final working behaviour). The exploit was closed by modifying `_check_finish_line()` to return `False` unless at least 85% of the waypoints have been visited. With this change the shortcut is effectively worthless, and cars must traverse the full circuit to receive a meaningful finish bonus.

4.1.2 Parameters

NEAT configuration. The following values were used in the final training run (`config/neat_waypoints.cfg`):

Parameter	Value
Population size	100
Fitness criterion	max
Fitness threshold	100 000
No fitness termination	False
Reset on extinction	0
Initial connection	full
Feed forward	True
Number of inputs	5
Number of outputs	2
Number of hidden nodes (initial)	0
Activation default	tanh
Activation mutate rate	0.05
Activation options	tanh, sigmoid, relu
Weight init mean / stdev	0.0 / 1.0
Weight mutate power	0.4
Weight mutate rate	0.8
Bias mutate rate	0.7
Connection add probability	0.25
Connection delete probability	0.30
Node add probability	0.10
Node delete probability	0.15
Compatibility threshold	3.0
Max stagnation	25
Species elitism	2
Elitism (per species)	2
Survival threshold	0.2

NEAT was started from a fully connected minimal topology (5 inputs $\rightarrow$ 2 outputs, no hidden layers) and topology was allowed to grow through connection and node add mutations. The population of 100 was chosen as a balance between diversity and training speed.

Car dynamics. The physical parameters of the car were fixed across all experiments:

Parameter	Value
Maximum velocity	4.0 px/frame
Maximum reverse velocity	2.0 px/frame
Rotation velocity	4.0 °/frame
Acceleration gain	1.0
Friction coefficient	0.94 (per frame when throttle < 0.05)
Steering gain	2.5× rotation velocity
Off-track kill threshold	90 consecutive frames
Grace period at spawn	30 frames
Waypoint stuck threshold	900 frames
Circular motion detector	120-frame window, 20 px radius
Waypoint reach radius	40 px
Finish gate	≥ 85% of waypoints visited
Sensor noise σ	0.03
Actuator noise σ	0.05

4.1.3 Controller Performance

The final waypoints controller was trained with the command: `python src/train_unified.py -strategy waypoints -generations 100`.

Training progression. Fitness data was logged to `results/waypoints/logs/generations.csv`. The best fitness improved from 1 344.53 in generation 0 to 12 188.70 by generation 46, after which it plateaued in the 12 150–12 190 band for the remainder of the run (Figure 1). The average fitness followed a noisier trajectory, peaking at 3 493.43 in generation 46 and ending at 2 005.19 in generation 99. The high variance in average fitness (final standard deviation 3 154.38) reflects the active mutation pressure: new structural mutations often perform poorly before speciation and elitism refine them.

Network complexity. Topology augmentation was active but conservative. The best genome’s node count grew from the initial 2 (the output nodes) to a maximum of 4, and connection counts ranged from 1 to 10 (Figure 2). The best genomes at the end of the run typically contained 2–4 hidden nodes and 3–8 connections, confirming that NEAT found a compact solution for this control task.

Observed behaviour. The car accelerates from the start position, turns north toward the first waypoint, and follows the clockwise circuit. It brakes noticeably on sharp curves (particularly the tight hairpin at the bottom of the track) and accelerates again on the subsequent straights. The control is smooth rather than oscillatory: the network has learned to apply continuous moderate steering corrections rather than sharp alternating turns.

Robustness statistics (30 trials). The best-ever genome (key 4464, generation 46, training fitness 8 641.87) was evaluated over 30 independent

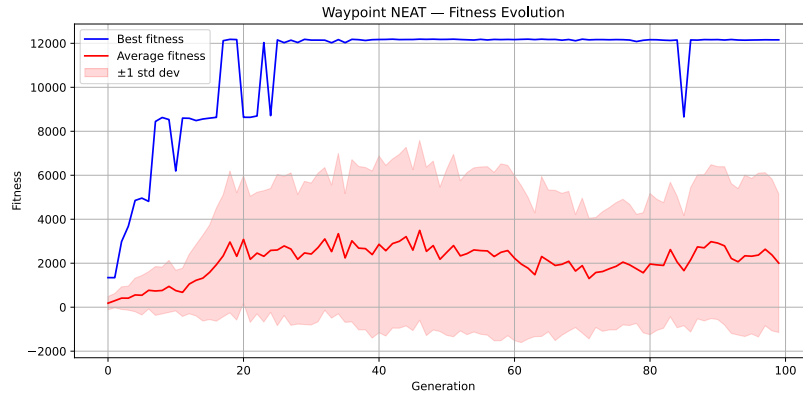


Figure 1: Waypoint NEAT fitness evolution over 100 generations. Best fitness (blue) plateaus around 12 150 from generation 17 onward. Average fitness (red dashed) and its ± 1 standard-deviation band show the population's exploratory variance.

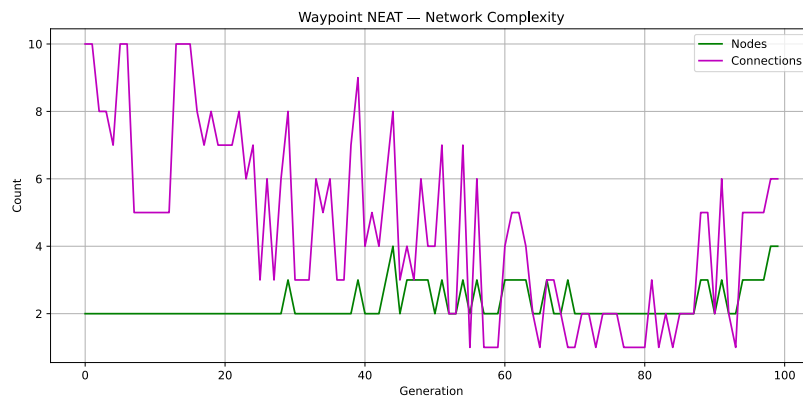


Figure 2: Network complexity of the best genome per generation. The topology grows modestly from 2 nodes / 10 connections to 4 nodes / 6 connections, indicating that the control policy does not require deep hierarchical processing.

demo trials using `demo_winner.py -final -trials 30`. Each trial used the same starting position but independent noise realisations, so outcome variance reflects genuine noise sensitivity.

Metric	Value
Total trials	30
Valid finishes (0 off-track)	10 (33.3%)
Dirty finishes (some off-track)	20 (66.7%)
Died before end	0 (0.0%)
Mean finish time (frames)	907
Min finish time	904
Max finish time	910
Mean off-track frames (finishers)	4.7
Min off-track frames	0
Max off-track frames	20

All 30 trials reached the finish line, demonstrating that the controller is highly reliable at completing the circuit. The majority of runs (66.7%) incurred brief off-track contact, typically 1–9 frames, with a few outliers reaching 20 frames. This suggests the car drives close to the edge on sharp curves, occasionally clipping the border without accumulating the 90 consecutive off-track frames required to die. A clean zero-off-track finish occurred in 33.3% of trials. Finish times are tightly clustered (904–910 frames, ≈ 15.1 – 15.2 s), indicating consistent speed.

Reverse-direction generalisation. A zero-shot generalisation test was performed by evaluating the winner on the reversed waypoint path (`PATH[::-1]`) without retraining. The car successfully completed the counter-clockwise circuit, reaching all 22 waypoints and crossing the finish line in 917 frames. This indicates that the controller has learned a robust geometric policy rather than simply memorising the clockwise waypoint sequence.

4.1.4 Different Race Configurations

Reverse direction. The reversed-circuit test described above constitutes the primary different-race experiment. Because the waypoints car receives absolute waypoint coordinates, one might expect it to fail on the reversed path without retraining. Instead, the car finished successfully, suggesting that the network learned to interpret relative geometric cues (distance, angle, curve sharpness) rather than memorising a fixed sequence of (x, y) coordinates.

Different finish-line positions.

4.1.5 Different Race Configurations

To test whether the controller generalises beyond the waypoint layout it was trained on, a second track configuration was defined with `setup_track.py`: 32 waypoints tracing a modified racing line, finish-line centre at (498, 482)

with a 90 rotation, and car spawn at (525, 477) heading 112.5. The winning genome from the primary 100-generation run was evaluated on this new layout without any retraining.

Metric	Value
Total trials	30
Valid finishes (0 off-track)	0 (0.0%)
Dirty finishes (some off-track)	30 (100.0%)
Died before end	0 (0.0%)
Mean finish time (frames)	1 020
Min / Max finish time	924 / 1 858
Mean off-track frames (finishers)	23.8
Min / Max off-track frames	13 / 61

The car completed the circuit in every trial, confirming it was not simply memorising the original waypoint coordinates. However, no trial was clean: every run incurred between 13 and 61 off-track frames, compared to 0–20 on the training layout. Three anomalous trials (8, 15, 16) registered approximately 1 850 frames and $wp = 63/32$ — roughly double the expected values — indicating the car completed an additional full lap before crossing the finish line on those runs. The higher off-track contact on the new layout is expected: the controller was optimised for the curvature profile of the 22-waypoint training path, and the new path introduces sharper or differently-spaced curves that push the car closer to the border. These results suggest the learned policy captures genuine geometric cues (distance and angle to waypoint, curve sharpness) rather than a memorised sequence, though fine-tuned boundary-hugging behaviour does not transfer perfectly to an unseen layout.

4.2 Radars Car Analysis

4.2.1 Experiments

A simulation is run for N generations, where in each generation, the total number of cars of each population race in the `pygame` track.

For each step, if the car is either out of bounds, in the track border, in the grass or reaches the finish line, it is considered not alive and its execution ends.

For each step of a car, its fitness is computed based on a fitness function defined in the app config file, so that its performance can be compared with the others.

Sadly, the only analyzed fitness function is a very basic one based on the total distance traveled by the car, which guarantees that cars that survive for longer without hitting the track borders are considered better, and the car’s corresponding neural network survives longer.

No other fitness functions or parameters were tested due to time limits. The car is not always able to do the best turns due to the simplicity of this fitness function. Further investment in fitness functions is required.

The winner network of the last generation is saved and can be reused in posterior runs by setting the radar car train to false in the app config file. Only

the last one is saved because the best n cars of each generation are always saved to the next generation.

4.2.2 Parameters

The parameters of the radar car can be configured in the config.ini file in the neat_cars/radars_car directory. I will cover here the most relevant ones.

The radar car has 5 **input nodes**, 3 **output nodes** and 0 **hidden nodes** at the start, which are all connected at the start (**initial_connection** is full_direct).

The input nodes are the radar distances, the output nodes are the velocity (bounded at the car's max velocity) and 2 integer booleans that define if the car should turn left, right or keep going straight.

The outputs follow a **tanh activation function**, meaning their values are between -1 and 1 . To transform these values according to the car's needs:

- The velocity is $abs(output) \times max_velocity$ so that it stays in the range $[0, max_velocity]$;
- The turning decision is:
 - Do not turn if both left and right are on;
 - Turn left if output > 0 ;
 - Turn right if output < 0 ;
 - Do not turn if both left and right are off.

The **number of generations** is set to 50 just to avoid very long training sessions, but can be configured in the app config file.

Still, the **fitness_criterion** is set to 100000 and **no_fitness_termination** to False to stop early in case some car of the current generation reaches that fitness value.

The population (**pop_size**) is 300 to possibly obtain better results, but with the increasing computational cost and time.

The **aggregation** of the input signals in each node is made by the sum function.

The **compatibility_threshold** is set to 3.0 so that the cars with similar performance are grouped together in the same species.

In terms of connections and nodes:

- **conn_add_prob** is 0.4
- **conn_delete_prob** is 0.4
- **node_add_prob** is 0.15
- **node_delete_prob** is 0.15

In terms of species:

- **species_fitness_func** uses the maximum fitness of each species to know which species to preserve;
- **max_stagnation** is 20 so that species that do not improve for 20 generations are removed;

- **species_elitism** is 5 so that the best 5 species are not removed.

The **elitism** is set to 5 so that the best 5 genomes are preserved in the next generation.

The car's parameters (**max_vel** and **rotation_vel**) are fixed at 4.

4.2.3 Controller Performance

The simulation can be watched while it happens in `pygame`, and the results of each generation can be checked in the terminal while it runs.

After the testing is over, the pruned and unpruned winner neural network can be visualized in the directory set in the app config file, along with a graph with the fitness stats and the species stats along the generations.

During the true race, the current velocity, acceleration, angle and fitness of the radar car can be seen in the `pygame` UI.

4.3 Performance Comparison of the Cars

A direct quantitative comparison between the two strategies is not trivial: In reality, the two approaches represent different points on a *knowledge vs. adaptability* trade-off:

- The **waypoints car** has access to rich, task-specific information (distance, heading error, curve geometry) that makes the learning problem considerably easier. Fitness improves quickly and the required network is small. The controller is, however, tied to the specific waypoint layout: it will fail on a different track or reversed direction without re-training.
- The **radar car** must infer track geometry from raw distance readings, which is a harder credit-assignment problem and likely requires more generations. In exchange, the resulting controller generalises better to unseen track segments because it responds to local geometry rather than memorised waypoint coordinates.